

The Islamia University of Bahawalpur
Department of Computer Science

MERN Laptop E-commerce Platform

Software Requirements Specification & System Design Document

Prepared By:

Muhammad Hammad

F22BDOCS1M1094

Session: Spring/Fall 2022 – 2026

Supervisor:

Mam. Sadiqa Ajmal

Bachelor of Science in Computer Science

Version: 1.0

Date: 09/12/2025

Revision History

Date	Description	Author	Comments
09/12/2025	Initial Draft Created	Muhammad Hammad	First Version
20/12/2025	SRS Complete	Muhammad Hammad	All requirements documented
05/01/2026	SDD Added	Muhammad Hammad	Architecture and design
15/01/2026	Final Version	Muhammad Hammad	Complete SRS/SDD

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Mam. Sadiqa Ajmal	Supervisor, CSIT 21306	09/12/2025

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Scope	5
1.3	Definitions, Acronyms, and Abbreviations	6
1.4	References	7
1.5	Overview	8
1.6	Product Perspective	8
	1.6.1 Operations	8
	1.6.2 Site Adaptation Requirements	9
1.7	Product Functions	9
1.8	User Characteristics	9

1.9	Assumptions and Dependencies	10
2	Software Requirements Specification	11
2.1	Specific Requirements	11
2.2	External Interfaces	11
2.3	Functions	11
2.4	Performance Requirements	12
2.5	Logical Database Requirements	12
2.6	Design Constraints	12
2.6.1	Standards Compliance	12
2.7	Software System Attributes	13
2.7.1	Reliability	13
2.7.2	Availability	13
2.7.3	Security	13
2.7.4	Maintainability	13
2.7.5	Usability	13
2.8	Alternative Architectures Considered	13
2.8.1	Monolithic PHP/Laravel Architecture (Not Selected)	13
2.8.2	Microservices Architecture (Not Selected)	14
2.8.3	Final Justification	14
2.9	Use Cases	14
2.9.1	Use Case 1: Customer Registration and Login	14
2.9.2	Use Case 2: Product Browsing and Filtering	15
2.9.3	Use Case 3: Shopping Cart Management	15
2.9.4	Use Case 4: Order Processing and Payment	16
2.9.5	Use Case 5: Admin Product Management	17
2.10	Classes and Objects	17
2.10.1	User Class	17
2.10.2	Product Class	18
2.10.3	Order Class	19
2.10.4	Cart Class	19
3	System Design and Architecture	21
3.1	Architectural Design	21
3.2	Decomposition Description	21
3.3	Component Design	22
3.3.1	Authentication Component	22
3.3.2	Cart Calculation Component	22
3.3.3	Order Processing Component	23
3.4	Data Design	24
3.4.1	Data Description	24
3.4.2	Database Schemas	24

4	Implementation and Testing	27
4.1	Technology Stack	27
4.2	API Specifications	27
4.2.1	REST API Endpoints	27
4.3	Design System	28
4.3.1	Color Palette	28
4.3.2	Typography	29
4.3.3	Spacing & Layout	29
4.3.4	Components	30
4.4	Screen Images	30
4.5	Test Cases	32
4.6	Deployment Configuration	34
4.6.1	Environment Variables	34
4.7	Security Testing Results	34
4.8	Disaster Recovery Procedures	35
4.9	Conclusion	35

Chapter 1

1 Introduction

The MERN Laptop E-commerce Platform is a sophisticated, full-featured online store specifically designed to address the unique needs of customers shopping for laptops and the businesses selling them. In today's digital marketplace, laptop buyers require detailed technical information, powerful filtering capabilities, and a seamless purchasing experience. Traditional generic e-commerce platforms often fail to provide the specialized features needed for laptop sales, such as filtering by RAM, ROM, processor generation, and other technical specifications that are critical to informed purchasing decisions.

This platform moves beyond conventional e-commerce solutions by offering an intuitive, feature-rich shopping experience built on the modern MERN (MongoDB, Express.js, React.js, Node.js) technology stack. The system provides customers with the ability to browse a comprehensive catalog of laptops, filter products based on detailed technical specifications, manage shopping carts, process orders securely through PayPal or Cash on Delivery, and track order status. For administrators, the platform includes a powerful dashboard for product management, inventory control, order processing, and user management.

Security and trust are paramount in e-commerce. The platform implements industry-standard security measures including JWT-based authentication, password hashing with bcrypt, HTTPS encryption for all data transmission, and secure integration with PayPal's payment gateway. All user data is protected, and transaction processing follows best practices for financial security.

The platform is designed to be scalable, maintainable, and performant, capable of handling 1000+ concurrent users during peak shopping periods. With a focus on responsive design using Tailwind CSS, the platform provides an optimal shopping experience across all devices – from desktop computers to smartphones and tablets.

1.1 Purpose

The purpose of the MERN Laptop E-commerce Platform project is to develop a specialized, full-featured online store that addresses the specific needs of customers shopping for laptops and the businesses selling them. This project aims to move beyond generic e-commerce solutions by providing detailed technical filtering capabilities (RAM, ROM, generation, processor type, graphics card, etc.), a secure and seamless purchasing journey, and robust administrative tools for inventory and order management.

The platform leverages the modern MERN stack to deliver a fast, scalable, and maintainable application that builds customer trust through reliable transactions (via PayPal

integration) and a professional user experience. The ultimate goal is to create a complete e-commerce ecosystem that drives online sales for laptop retailers while providing customers with an enjoyable, efficient, and secure shopping experience.

Key objectives of the platform include:

For Customers:

- Easy product discovery through advanced filtering and search capabilities
- Detailed product information including complete technical specifications
- Secure and convenient payment options (PayPal and Cash on Delivery)
- Order tracking and history management
- User-friendly interface with responsive design for all devices

For Administrators:

- Comprehensive product management with image upload to Cloudinary
- Real-time inventory tracking and stock management
- Order processing and status management
- User account management and role-based access control
- Sales monitoring and business insights

1.2 Scope

The MERN Laptop E-commerce Platform is a comprehensive web application that falls within the category of Web Application based Information System with e-commerce capabilities. The platform provides complete online shopping functionality for laptop products with separate interfaces for customers and administrators.

The system will provide customers with the following core capabilities:

User Registration and Authentication: Secure account creation with email validation, login functionality using JWT tokens, password recovery, and profile management. Users can update their personal information, manage shipping addresses, and view order history.

Product Browsing with Advanced Filtering: Comprehensive laptop catalog browsing with filtering by RAM (4GB, 8GB, 16GB, 32GB+), ROM/storage (256GB SSD, 512GB SSD, 1TB SSD, 1TB+ HDD), processor generation (Intel Core i3/i5/i7/i9, AMD Ryzen 3/5/7/9), brand (Dell, HP, Lenovo, Apple, ASUS, Acer, MSI), price range, color, weight, and screen size. Sorting options include price (low to high, high to low), popularity, newest arrivals, and customer ratings.

Shopping Cart Management: Add products to cart, update quantities, remove items, and view real-time subtotal calculations. Cart persistence allows users to save their cart for future sessions. Stock availability validation prevents overselling.

Order Processing and Payment Integration: Secure checkout process with shipping address collection, payment method selection (PayPal integration and Cash on Delivery), order confirmation, and email notifications. Complete order history with status tracking (Pending, Processing, Shipped, Delivered, Cancelled).

Admin Dashboard: Secure administrative interface for product management (add, edit, delete products, manage images via Cloudinary), order processing (view orders, update status), user management (view users, manage roles), and inventory control (track stock levels, receive low stock alerts).

The system will NOT include the following features in this version:

- Voice search functionality
- Augmented reality (AR) product visualization
- Advanced AI-based product recommendations
- Multi-vendor marketplace support
- International shipping calculations
- Live chat or chatbot support
- Mobile native applications (though responsive web app is supported)
- Product comparison tool (planned for future version)

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
MERN	MongoDB, Express.js, React.js, Node.js technology stack for full-stack JavaScript development
JWT	JSON Web Token - A compact, URL-safe means of representing claims to be transferred between two parties for authentication
CRUD	Create, Read, Update, Delete - The four basic operations of persistent storage
SRS	Software Requirements Specification - A document describing the expected features, functionality, and constraints
SDD	Software Design Document - A document describing system architecture, components, and design decisions
API	Application Programming Interface - A set of rules for building and interacting with software applications

REST	Representational State Transfer - An architectural style for designing networked applications
HTTPS	Hypertext Transfer Protocol Secure - An extension of HTTP for secure communication
JSON	JavaScript Object Notation - A lightweight data-interchange format
UI	User Interface - The visual elements and interactive components
ODM	Object Document Mapper - Mongoose for MongoDB data modeling
COD	Cash on Delivery - Payment method where customer pays upon delivery
RAM	Random Access Memory - Computer memory specification for laptops
ROM	Read-Only Memory / Storage - Storage capacity specification for laptops

1.4 References

1. React.js Official Documentation: <https://react.dev/>
2. Node.js Official Documentation: <https://nodejs.org/en/docs/>
3. Express.js Official Documentation: <https://expressjs.com/>
4. MongoDB Official Documentation: <https://docs.mongodb.com/>
5. Mongoose ODM Documentation: <https://mongoosejs.com/docs/>
6. PayPal Developer Documentation: <https://developer.paypal.com/>
7. Cloudinary API Documentation: <https://cloudinary.com/documentation>
8. Tailwind CSS Documentation: <https://tailwindcss.com/docs>
9. JWT.io - JSON Web Token Introduction: <https://jwt.io/introduction/>
10. IEEE Standard 830-1998 - Recommended Practice for Software Requirements Specifications
11. IEEE Standard 1016-2009 - Recommended Practice for Software Design Descriptions
12. WCAG 2.1 Web Accessibility Guidelines: <https://www.w3.org/WAI/WCAG21/quickref/>

1.5 Overview

The MERN Laptop E-commerce Platform is a full-stack web application built with MongoDB, Express.js, React.js, and Node.js. It features a responsive React frontend styled with Tailwind CSS and a secure Node.js/Express backend REST API. The system is structured around two core interfaces: a customer-facing store for browsing, filtering, and purchasing laptops, and an admin dashboard for managing products, orders, and users. Key integrations include PayPal for payment processing and Cloudinary for image management. All user authentication is handled via JWT tokens, and data is persisted in a MongoDB database using Mongoose ODM.

This document is organized into four chapters. Chapter 1 introduces the system, its purpose, scope, and context within the e-commerce ecosystem. Chapter 2 details the specific software requirements, including external interfaces, functional and non-functional requirements, use cases with detailed flows, class and object definitions, and logical database requirements. Chapter 3 presents the system design and architecture, including architectural decisions, alternative architectures considered, component design, data design, and complete data dictionary with all MongoDB schemas. Chapter 4 covers implementation and testing considerations, including technology stack, API specifications, design system with color palette and typography, screen images with placeholders, and comprehensive test cases.

1.6 Product Perspective

The MERN Laptop E-commerce Platform is an independent, self-contained web application designed to provide complete online shopping capabilities for laptop products. Unlike generic e-commerce platforms that require extensive customization for laptop-specific features, this product is purpose-built with specialized filtering options based on technical specifications (RAM, ROM, generation, processor, etc.).

The system will operate as a full-stack MERN application with React.js frontend, Node.js/Express.js backend REST API, and MongoDB database for persistent storage. The product emphasizes user experience, security, and comprehensive administrative controls for efficient business operations. All communication between frontend and backend occurs through RESTful APIs with JWT authentication for protected routes.

1.6.1 Operations

The system will support continuous e-commerce operations with real-time inventory management. Normal operations include user registrations, product browsing, shopping cart management, order processing, payment transactions, and administrative controls. The system will maintain 24/7 availability with secure transaction processing.

Data backup operations will preserve customer information, product catalog, and order history. Automated daily backups ensure data integrity and disaster recovery capability. The system will support high concurrent user loads during peak shopping periods with optimized database queries, proper indexing, and caching mechanisms.

1.6.2 Site Adaptation Requirements

The system requires a web hosting environment with Node.js runtime (version 18+), MongoDB database (version 6+), and SSL certification for secure HTTPS communication. Installation will require configuration of PayPal payment gateway (API credentials), Cloudinary image storage (cloud name, API key, API secret), JWT authentication (secret key), and database connection string.

Browser compatibility will be ensured for major modern web browsers including Google Chrome (version 90+), Mozilla Firefox (version 88+), Apple Safari (version 14+), and Microsoft Edge (version 90+). Mobile device access will be supported through responsive design using Tailwind CSS breakpoints.

1.7 Product Functions

The core functionality of the MERN Laptop E-commerce Platform centers around comprehensive online shopping operations with specialized laptop features:

User Authentication System: The system enables customer registration with email validation, secure login using JWT tokens with 7-day expiration, password recovery functionality, and profile management. Role-based access control differentiates between customer and administrator privileges.

Product Management System: Comprehensive laptop product catalog management with detailed specifications including image upload to Cloudinary, title, generation, color, RAM (GB), ROM (GB), weight (kg), description, category, brand, price, sale price, total stock, and average review rating.

Advanced Filtering and Search: Customers can browse products with filtering by category, brand, RAM (4GB/8GB/16GB/32GB+), ROM/storage (256GB/512GB/1TB+), processor generation (i3/i5/i7/i9, Ryzen 3/5/7/9), price range, and more. Sorting options include price (ascending/descending), popularity, and newest arrivals.

Shopping Cart Management: Add products to cart with quantity selection, update quantities, remove items, view real-time subtotal calculations, and persistent cart storage across sessions. Stock availability validation prevents overselling.

Order Processing System: Complete order lifecycle management from cart to delivery with shipping address collection, payment method selection (PayPal integration and Cash on Delivery), order confirmation emails, and status tracking (Pending, Processing, Shipped, Delivered, Cancelled).

Admin Dashboard: Secure administrative interface providing product management (CRUD operations), order processing (view, update status, cancel), user management (view, role management), inventory tracking (stock levels, low stock alerts), and sales analytics.

1.8 User Characteristics

The primary user groups for the platform are customers and administrators, each with distinct characteristics and requirements:

Customers (Laptop Shoppers): These are individuals seeking to purchase laptops

online. Their technical proficiency varies from basic to advanced. They require intuitive product discovery, easy filtering by technical specifications, detailed product information, simple cart management, and secure payment options. Customers value responsive interfaces, fast loading times, and reliable order tracking. They expect a seamless shopping experience from product discovery to delivery confirmation.

Administrators (Business Staff): These are employees of the laptop store responsible for managing the online platform. They have moderate to advanced technical skills and require efficient dashboard interfaces for product management, order processing, user management, and inventory control. Administrators need quick access to business insights, real-time stock monitoring, and the ability to process orders efficiently. They value comprehensive reporting, easy product updates, and streamlined order management workflows.

1.9 Assumptions and Dependencies

The platform development assumes the availability of MERN stack technologies (MongoDB, Express.js, React.js, Node.js) and third-party service integrations. The system depends on stable internet connectivity for payment processing through PayPal API and image management through Cloudinary API.

External service availability and API rate limits may impact functionality. PayPal transaction processing depends on PayPal service uptime and API responsiveness. Cloudinary image uploads depend on service availability and network conditions.

The platform assumes that customers have modern web browsers with JavaScript enabled and that administrators have basic computer literacy. The system assumes that product inventory data is maintained accurately and that payment transactions are processed in compliance with financial regulations.

Chapter 2

2 Software Requirements Specification

2.1 Specific Requirements

The MERN Laptop E-commerce Platform shall provide secure user authentication using JWT tokens, comprehensive product management with specialized laptop filtering, shopping cart functionality with real-time calculations, order processing with PayPal and COD payment options, and an administrative dashboard for complete business management. All requirements are uniquely identified and shall be verifiable through the test cases defined in Chapter 4.

2.2 External Interfaces

1. **Customer Storefront:** React.js application with responsive design, product catalog with filtering, shopping cart, and checkout interface
2. **Admin Dashboard:** Secure administrative interface for product management, order processing, and user management
3. **PayPal Integration:** Payment gateway API for secure transaction processing
4. **Cloudinary Integration:** Image storage and delivery API for product images
5. **REST API:** Backend services accessible through RESTful endpoints with JWT authentication

2.3 Functions

The system shall validate user credentials and manage sessions using JWT tokens with secure password hashing via bcrypt. The system shall provide product CRUD operations with image upload to Cloudinary. The system shall support advanced filtering by RAM, ROM, generation, brand, price range, and other specifications. The system shall maintain shopping cart persistence and real-time calculations. The system shall process orders through PayPal API with secure transaction handling. The system shall provide admin interfaces for complete business management.

2.4 Performance Requirements

Requirement	Target Metric
Product Catalog Loading Time	≤ 3 seconds on standard broadband
Search and Filter Results	≤ 2 seconds
Payment Processing Time	≤ 10 seconds including PayPal API
Concurrent User Support	1000+ concurrent users
Database Query Performance	Optimized with indexing for sub-second response
API Response Time	≤ 500ms for non-payment endpoints

2.5 Logical Database Requirements

The platform will use MongoDB as the primary database with collections for users, products, orders, carts, addresses, features, and product reviews. All collections will use Mongoose schemas with validation. Relationships between collections will be maintained through referenced ObjectIds. Indexing will be applied to frequently queried fields including email (users), category (products), brand (products), price (products), and order status (orders). Data validation will be enforced at the schema level to ensure data integrity.

2.6 Design Constraints

Development must adhere strictly to the MERN technology stack (MongoDB, Express.js, React.js, Node.js). The frontend must implement responsive design using Tailwind CSS for full mobile compatibility. Integration with PayPal for payments and Cloudinary for image storage is mandatory. Security standards including JWT authentication, bcrypt password hashing, and HTTPS encryption must be followed throughout the application.

2.6.1 Standards Compliance

- MERN stack development best practices
- RESTful API design principles
- Web Content Accessibility Guidelines (WCAG 2.1 Level A)
- Payment Card Industry Data Security Standards (PCI DSS via PayPal)
- MongoDB schema design best practices
- React.js component architecture standards

2.7 Software System Attributes

2.7.1 Reliability

System uptime shall maintain 99.9% availability through cloud deployment. Transaction processing shall ensure data consistency with rollback capabilities. Automated daily backups shall maintain data integrity and disaster recovery readiness.

2.7.2 Availability

The system shall maintain 24/7 availability through cloud hosting. Database replication shall ensure high availability during maintenance windows. Graceful degradation shall occur during high load periods.

2.7.3 Security

JWT-based authentication with 7-day token expiration. Bcrypt password hashing with 10 salt rounds. HTTPS encryption for all data transmission. Role-based access control for admin routes. Input validation and sanitization to prevent injection attacks.

2.7.4 Maintainability

Modular code structure with separation of concerns. Comprehensive API documentation. Version control with Git. Automated testing suite. Environment-based configuration management.

2.7.5 Usability

Responsive design for desktop, tablet, and mobile devices. Intuitive navigation and search functionality. Clear error messages and user feedback. Accessible interface with WCAG 2.1 Level A compliance.

2.8 Alternative Architectures Considered

2.8.1 Monolithic PHP/Laravel Architecture (Not Selected)

This architecture would have used PHP with Laravel framework for both frontend and backend. It was rejected because:

- Does not meet the MERN stack requirement for the project
- Less suitable for real-time cart updates and dynamic UI
- Steeper learning curve for modern JavaScript features
- Less flexible for future mobile app extensions

2.8.2 Microservices Architecture (Not Selected)

This architecture would have decomposed the system into multiple independent services. It was rejected because:

- Adds unnecessary complexity for the project scope
- Requires sophisticated orchestration and deployment
- Higher operational overhead for academic project
- Development time would exceed project constraints

2.8.3 Final Justification

The MERN stack layered architecture was selected because:

- Provides full-stack JavaScript development consistency
- Offers excellent performance with React's virtual DOM
- MongoDB provides flexible schema for product specifications
- Node.js/Express provides efficient API handling
- Enables code reuse between frontend and backend
- Large ecosystem of libraries and community support

2.9 Use Cases

2.9.1 Use Case 1: Customer Registration and Login

Actor: Customer

Precondition: Customer accesses the platform for shopping

Main Flow:

1. Customer accesses registration page
2. Enters personal details (username, email, password)
3. System validates and creates user account
4. Customer receives confirmation via email
5. Customer logs in with credentials
6. System authenticates and creates JWT session
7. Customer accesses personalized shopping features

Postcondition: Customer account created and authenticated

Figure 1: Customer Registration and Login

[Placeholder: Use Case Diagram Image]

Actors: Customer, System; Actions: Register, Login, Authenticate

2.9.2 Use Case 2: Product Browsing and Filtering

Actor: Customer

Precondition: Customer is logged in and browsing products

Main Flow:

1. Customer accesses product catalog
2. Applies filters (RAM, ROM, generation, brand, price)
3. Views filtered results in real-time
4. Sorts products by price, popularity, or newest
5. Views product details and specifications
6. Checks product availability and reviews
7. Adds desired products to cart

Postcondition: Customer finds desired laptop products

2.9.3 Use Case 3: Shopping Cart Management

Actor: Customer

Precondition: Customer has selected products for purchase

Main Flow:

1. Customer adds products to cart
2. System updates cart contents and displays
3. Customer reviews cart items
4. Updates quantities or removes items as needed
5. System recalculates totals in real-time

6. System validates stock availability
7. Customer proceeds to checkout when ready

Postcondition: Cart prepared for checkout

2.9.4 Use Case 4: Order Processing and Payment

Actor: Customer

Precondition: Customer has items in cart ready for purchase

Main Flow:

1. Customer initiates checkout process
2. Selects or adds shipping address
3. Selects delivery method
4. Chooses payment method (PayPal or COD)
5. For PayPal: redirects to PayPal payment gateway
6. For COD: confirms order without payment
7. System validates payment and processes order
8. System confirms order and updates inventory
9. Customer receives order confirmation email
10. Customer can track order status from profile

Postcondition: Order successfully placed and confirmed

Figure 2: Order Processing and Payment

[Placeholder: Sequence Diagram Image]

Customer → Cart → Checkout → Payment → Order Confirmation

2.9.5 Use Case 5: Admin Product Management

Actor: Administrator

Precondition: Admin is logged into dashboard with admin role

Main Flow:

1. Admin accesses product management section
2. Adds new laptop products with complete specifications
3. Uploads product images to Cloudinary
4. Sets pricing (regular price, sale price)
5. Sets inventory levels (total stock)
6. Manages product categories and brands
7. Updates product information as needed
8. Removes discontinued products

Postcondition: Product catalog updated and managed

2.10 Classes and Objects

2.10.1 User Class

Description: Represents system users including customers and administrators.

Attributes:

Attribute	Data Type	Description
_id	ObjectId	Unique identifier (MongoDB auto-generated)
userName	String	Unique username for login
email	String	Unique email address for contact and authentication
password	String	Hashed password using bcrypt
role	String	User role ('user' for customers, 'admin' for administrators)
resetPasswordToken	String	Token for password reset functionality
resetPasswordExpires	Date	Expiration timestamp for reset token

Methods:

Method	Parameters	Description
authenticate()	email, password	Verifies user credentials and returns JWT token
updateProfile()	userId, updatedData	Updates user profile information

manageAddresses()	userId, addressData	Adds, updates, or removes shipping addresses
viewOrderHistory()	userId	Retrieves all orders placed by user
resetPassword()	email, token, newPassword	Resets user password with valid token

2.10.2 Product Class

Description: Represents laptop products available for purchase with detailed technical specifications.

Attributes:

Attribute	Data Type	Description
_id	ObjectId	Unique identifier (MongoDB auto-generated)
image	String	Cloudinary URL for product image
title	String	Product name/title
generation	String	Processor generation (e.g., "Intel Core i7-12th Gen")
color	String	Product color
ram	Number	RAM size in GB (4, 8, 16, 32+)
rom	Number	Storage size in GB (256, 512, 1024+)
weight	Number	Product weight in kg
description	String	Detailed product description
category	String	Product category (Gaming, Ultrabook, Convertible, Chromebook)
brand	String	Laptop brand (Dell, HP, Lenovo, Apple, ASUS, Acer, MSI)
price	Number	Regular product price
salePrice	Number	Discounted sale price (if applicable)
totalStock	Number	Available inventory quantity
averageReview	Number	Average customer rating (0-5)
createdAt	Date	Product creation timestamp
updatedAt	Date	Last product update timestamp

Methods:

Method	Parameters	Description
updateStock()	productId, quantity	Updates inventory after order placement
addReview()	productId, reviewData	Adds customer review and updates average rating
applyFilter()	filterCriteria	Filters products by specifications
calculateDiscount()	price, salePrice	Calculates discount percentage

2.10.3 Order Class

Description: Represents customer orders including items, shipping, payment, and status.

Attributes:

Attribute	Data Type	Description
_id	ObjectId	Unique identifier (MongoDB auto-generated)
userId	String	Reference to ordering user
cartId	String	Reference to source cart
cartItems	Array	Order items with product details, quantity, price
addressInfo	Object	Shipping address details
orderStatus	String	Status (Pending, Processing, Shipped, Delivered, Cancelled)
paymentMethod	String	Payment method (PayPal, COD)
paymentStatus	String	Payment status (Pending, Paid, Failed, Refunded)
totalAmount	Number	Order total amount
orderDate	Date	Order creation timestamp
orderUpdateDate	Date	Last status update timestamp
paymentId	String	PayPal transaction ID (if applicable)
payerId	String	PayPal payer ID (if applicable)

Methods:

Method	Parameters	Description
calculateTotal()	cartItems	Computes order total from items
updateStatus()	orderId, newStatus	Updates order status and triggers notifications
generateInvoice()	orderId	Creates invoice PDF for order
trackShipping()	orderId	Retrieves shipping status and tracking number

2.10.4 Cart Class

Description: Represents user shopping cart with items and quantities.

Attributes:

Attribute	Data Type	Description
_id	ObjectId	Unique identifier (MongoDB auto-generated)
userId	ObjectId	Reference to cart owner
items	Array	Cart items with productId and quantity
createdAt	Date	Cart creation timestamp
updatedAt	Date	Last cart update timestamp

Methods:

Method	Parameters	Description
addItem()	productId, quantity	Adds product to cart with quantity
removeItem()	productId	Removes product from cart
updateQuantity()	productId, newQuantity	Updates quantity for cart item
calculateTotal()	None	Computes cart total from items
clearCart()	None	Removes all items from cart

Chapter 3

3 System Design and Architecture

3.1 Architectural Design

The MERN Laptop E-commerce Platform employs a layered architecture with three main tiers: Presentation Layer, Business Logic Layer, and Data Layer, following the Model-View-Controller (MVC) pattern adapted for the MERN stack. This separation ensures modularity, testability, and maintainability while supporting the requirements of e-commerce operations.

Figure 3: System Architecture Overview

[Placeholder: Layered Architecture Diagram]

Presentation (React.js) → API Gateway → Business Logic (Node.js/Express) → Data (MongoDB)

3.2 Decomposition Description

The system is decomposed into five main modules following modular design principles:

1. Authentication & User Management Module: Handles customer and admin registration, login, session management with JWT tokens, and role-based access control. Implements bcrypt password hashing for security.

2. Product Management Module: Manages laptop product catalog with detailed specifications (RAM, ROM, generation, brand, etc.), advanced filtering capabilities, image upload to Cloudinary, and real-time inventory tracking.

3. Shopping Cart Module: Provides shopping cart functionality with real-time calculations, persistent storage in MongoDB, stock availability validation, and seamless checkout transition.

4. Order Processing Module: Handles complete order lifecycle from cart to delivery with PayPal integration, Cash on Delivery option, order status tracking, and email notifications.

5. Admin Dashboard Module: Provides comprehensive administrative controls for product management, order processing, user management, inventory control, and business analytics.

3.3 Component Design

3.3.1 Authentication Component

Function: authenticateUser

Purpose: Authenticates user credentials and returns JWT token

Inputs: email, password

Local Data: userRecord, isValidPassword, jwtToken

Pseudocode:

```
BEGIN authenticateUser(email, password)
    userRecord      FIND user WHERE email = input email
    IF userRecord DOES NOT EXIST THEN
        RETURN "Invalid credentials"
    ENDIF

    isValidPassword  bcrypt.compare(password, userRecord.
        passwordHash)
    IF isValidPassword IS FALSE THEN
        RETURN "Invalid credentials"
    ENDIF

    jwtToken        jwt.sign({ id: userRecord.id, role: userRecord.
        role }, SECRET, { expiresIn: '7d' })
    RETURN { success: true, token: jwtToken, user: { id:
        userRecord.id, email: userRecord.email, role: userRecord.
        role } }
END
```

3.3.2 Cart Calculation Component

Function: calculateCartTotal

Purpose: Calculates total amount of items in shopping cart

Inputs: cartItems

Local Data: total, price, subtotal

Pseudocode:

```
BEGIN calculateCartTotal(cartItems)
    total          0
```

```

FOR EACH item IN cartItems DO
    price      item.product.salePrice OR item.product.price
    subtotal   price      item.quantity
    total      total + subtotal
END FOR

RETURN total
END

```

3.3.3 Order Processing Component

Function: processOrder

Purpose: Processes order with payment and inventory updates

Inputs: userId, cartItems, shippingAddress, paymentMethod

Local Data: orderItems, totalAmount, order

Pseudocode:

```

BEGIN processOrder(userId, cartItems, shippingAddress,
    paymentMethod)
    orderItems      []
    totalAmount      0

    FOR EACH item IN cartItems DO
        IF item.product.stock < item.quantity THEN
            RETURN "Insufficient stock for item: " + item.product
                .title
        ENDIF

        orderItems.push({
            productId: item.product.id,
            title: item.product.title,
            image: item.product.image,
            price: item.product.salePrice OR item.product.price,
            quantity: item.quantity
        })

        totalAmount      totalAmount + (item.product.price      item
            .quantity)
    END FOR

    order      CREATE order record with userId, orderItems,
        totalAmount, shippingAddress, paymentMethod

    FOR EACH item IN cartItems DO
        UPDATE product stock: item.product.stock - item.quantity
    END FOR

    CLEAR user cart

```



```

SEND order confirmation email to user

RETURN order

END

```

3.4 Data Design

3.4.1 Data Description

The system transforms e-commerce operations data into a MongoDB document-based structure with consideration of scalability and flexible product specifications. The design emphasizes document collections with proper indexing for query performance while maintaining data relationships through references.

Data storage is organized into seven main collections:

Collection	Description
users	Customer and admin authentication credentials, profiles, and role information
products	Laptop product catalog with detailed specifications, images, pricing, inventory
orders	Complete order information including items, shipping, payment, status tracking
carts	Shopping cart contents with persistent storage and real-time calculations
addresses	Customer shipping addresses for order delivery
features	Featured product promotions and banners
productreviews	Customer product reviews and ratings

3.4.2 Database Schemas

User Schema:

```

const UserSchema = new mongoose.Schema({
  userName: { type: String, required: true, unique: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, default: "user" },
  resetPasswordToken: { type: String },
  resetPasswordExpires: { type: Date }
});

```

Product Schema:

```

const ProductSchema = new mongoose.Schema({
  image: String,
  title: String,

```

```

    generation: String,
    color: String,
    ram: Number,
    rom: Number,
    weight: Number,
    description: String,
    category: String,
    brand: String,
    price: Number,
    salePrice: Number,
    totalStock: Number,
    averageReview: Number
  }, { timestamps: true });

```

Order Schema:

```

const OrderSchema = new mongoose.Schema({
  userId: String,
  cartId: String,
  cartItems: [{
    productId: String,
    title: String,
    image: String,
    price: String,
    quantity: Number
  }],
  addressInfo: {
    addressId: String,
    address: String,
    city: String,
    pincode: String,
    phone: String,
    notes: String
  },
  orderStatus: String,
  paymentMethod: String,
  paymentStatus: String,
  totalAmount: Number,
  orderDate: Date,
  orderUpdateDate: Date,
  paymentId: String,
  payerId: String
});

```

Cart Schema:

```

const CartSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User",
    required: true },

```

```
    items: [{
      productId: { type: mongoose.Schema.Types.ObjectId, ref: "
        Product", required: true },
      quantity: { type: Number, required: true, min: 1 }
    }]
  }, { timestamps: true });
```

Address Schema:

```
const AddressSchema = new mongoose.Schema({
  userId: String,
  address: String,
  city: String,
  pincode: String,
  phone: String,
  notes: String
}, { timestamps: true });
```

Chapter 4

4 Implementation and Testing

4.1 Technology Stack

Category	Technologies Used
Frontend Framework	React.js 18+ with functional components and hooks
UI Styling	Tailwind CSS for utility-first responsive styling
State Management	React Context API and Redux Toolkit
Routing	React Router DOM v6
Backend Framework	Node.js with Express.js REST API
Database	MongoDB with Mongoose ODM
Authentication	JWT (JSON Web Tokens) with bcrypt password hashing
Payment Gateway	PayPal SDK integration
Image Storage	Cloudinary API for upload and delivery
File Upload	Multer middleware for multipart form data
HTTP Client	Axios for API requests
Development Tools	Vite, ESLint, Prettier, Nodemon
Testing	Jest and React Testing Library

4.2 API Specifications

4.2.1 REST API Endpoints

Endpoint	Method	Auth	Description
/api/auth/register	POST	No	Register new user account
/api/auth/login	POST	No	Authenticate user and return JWT token
/api/auth/logout	POST	Yes	Invalidate user session
/api/products	GET	No	Get all products with filtering

/api/products/:id	GET	No	Get single product details
/api/products	POST	Admin	Create new product
/api/products/:id	PUT	Admin	Update product information
/api/products/:id	DELETE	Admin	Delete product
/api/cart	GET	Yes	Get user cart
/api/cart	POST	Yes	Add item to cart
/api/cart/:productId	PUT	Yes	Update cart item quantity
/api/cart/:productId	DELETE	Yes	Remove item from cart
/api/orders	POST	Yes	Create new order
/api/orders	GET	Yes	Get user orders
/api/orders/:id	GET	Yes	Get order details
/api/orders/:id/status	PUT	Admin	Update order status
/api/payment/create	POST	Yes	Create PayPal payment session
/api/payment/execute	POST	Yes	Execute PayPal payment
/api/addresses	GET	Yes	Get user addresses
/api/addresses	POST	Yes	Add new address

4.3 Design System

4.3.1 Color Palette

Primary (Green – Brand):

- Primary buttons, links, focus rings: hsl(142 71% 45%)
- Primary foreground (text on primary): #ffffff
- Tailwind classes: bg-primary, text-primary, ring-primary

Secondary: Secondary buttons, subtle backgrounds

Semantic Colors:

- Destructive: Errors, delete actions, danger states
- Muted: Backgrounds for less emphasis
- Accent: Hover states, highlights

Status Colors:

- In stock: text-emerald-600 / bg-emerald-500

- Low stock: text-amber-600 / bg-amber-500
- Out of stock: text-muted-foreground
- Sale: bg-tech-gradient

Tech Gradients:

- Main CTA: bg-tech-gradient (green → blue → purple)
- Category backgrounds: bg-gaming-gradient, bg-ultrabook-gradient, bg-convertible-gradient, bg-chromebook-gradient

4.3.2 Typography

Fonts:

- Sans/Display: Inter (var(--font-inter)) – headings and body
- Mono: JetBrains Mono – specifications and code

Typography Hierarchy:

Element	Class	Use
Page title	text-3xl md:text-4xl font-display font-bold	Hero, section titles
Section title	text-2xl md:text-3xl font-display font-bold	Section headings
Card title	text-lg font-semibold	Cards, list headers
Body	text-sm md:text-base	Paragraphs
Caption	text-sm text-muted-foreground	Labels, hints
Small	text-xs text-muted-foreground	Badges, meta

4.3.3 Spacing & Layout

Spacing Scale (Tailwind):

- 1 = 4px, 2 = 8px, 3 = 12px, 4 = 16px, 6 = 24px, 8 = 32px, 12 = 48px, 16 = 64px

Containers:

- Default: container mx-auto px-4 md:px-6
- Narrow (forms): max-w-md mx-auto
- Wide: max-w-6xl or container with screens.2xl: 1400px

Cards:

- Padding: p-4 (compact), p-6 (default)
- Gap between cards: gap-4 or gap-6
- Border radius: rounded-lg

4.3.4 Components

Buttons:

- Primary/CTA: bg-tech-gradient text-white
- Secondary: variant="secondary"
- Outline: variant="outline"
- Ghost: variant="ghost"
- Danger: variant="destructive"
- Sizes: sm, default, lg, icon

Forms:

- Inputs: rounded-lg and focus-visible:ring-ring
- Labels: text-sm font-medium
- Error: text-destructive text-sm

Navigation:

- Navbar: glass border-b border-border/50 sticky top-0 z-50
- Nav links: text-sm font-medium hover:text-primary transition-colors
- Active: text-primary or underline

4.4 Screen Images

Figure 4: Product Catalog / Home Page

[Placeholder: Product Listing with Filters]

Header with navigation, product grid, filtering sidebar, sorting options

Figure 5: Product Details Page

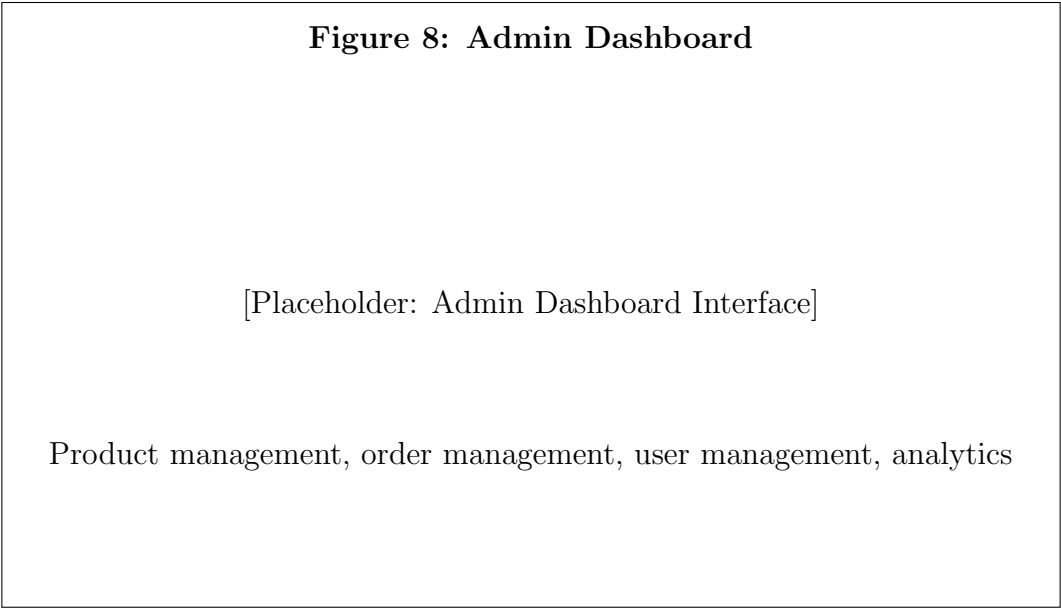
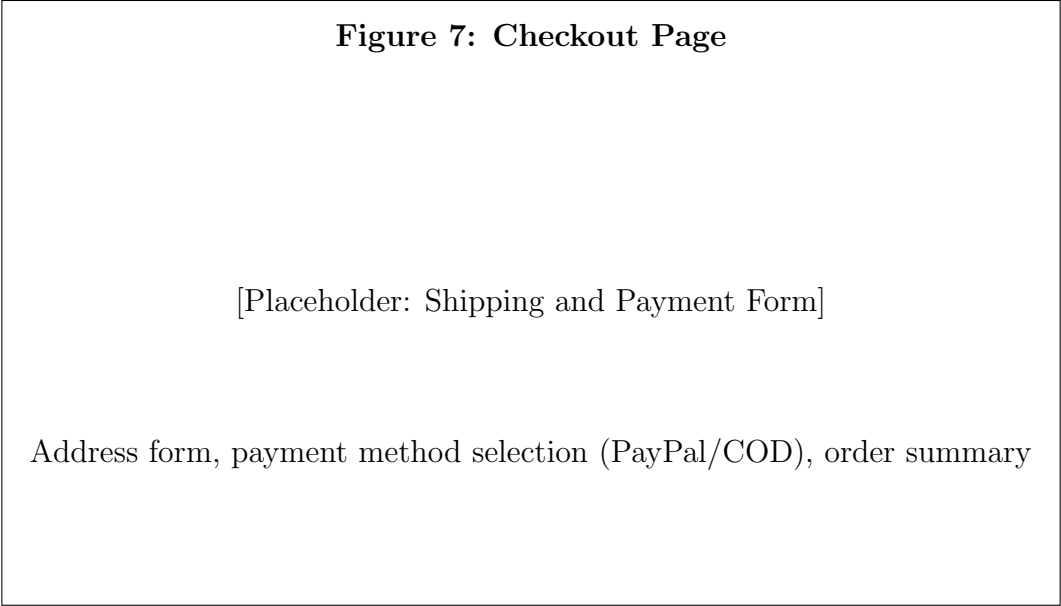
[Placeholder: Product Details with Specifications]

Product image, title, specifications, price, add to cart button, reviews

Figure 6: Shopping Cart

[Placeholder: Cart Items and Checkout]

Cart items with quantities, subtotal, proceed to checkout button



4.5 Test Cases

Test ID	Test Description	Input	Expected Output	Acceptance Criteria
TC-01	User Registration	Valid email, user-name, password	Account created, confirmation email sent	Registration completes within 3 seconds

TC-02	User Login	Valid credentials	JWT token returned, redirect to home	Login completes within 2 seconds
TC-03	Invalid Login	Wrong password	Error message displayed	System rejects invalid credentials
TC-04	Product Catalog Load	Access homepage	Product grid displayed with 10+ products	Loads within 3 seconds
TC-05	Filter by RAM	Select RAM 8GB	Products with 8GB RAM displayed	Filter results within 2 seconds
TC-06	Filter by Brand	Select Dell	Dell laptop products displayed	Filter results within 2 seconds
TC-07	Filter by Price Range	Select Rs. 50,000-100,000	Products in price range displayed	Filter results within 2 seconds
TC-08	Sort by Price Low-High	Click sort option	Products sorted ascending by price	Sort completes within 1 second
TC-09	Add to Cart	Click "Add to Cart"	Cart count increments, item added	Cart updates within 1 second
TC-10	Update Cart Quantity	Change quantity from 1 to 2	Cart total updates, stock validated	Update within 1 second
TC-11	Remove from Cart	Click remove icon	Item removed, cart total updates	Removal within 1 second
TC-12	Proceed to Checkout	Click "Checkout" button	Shipping address form displayed	Navigation within 2 seconds
TC-13	Add Shipping Address	Fill address form	Address saved, continue to payment	Address saves within 2 seconds
TC-14	PayPal Payment	Select PayPal, click Pay	Redirect to PayPal, payment processed	Transaction completes within 10 seconds
TC-15	COD Order	Select COD, place order	Order confirmed, status "Pending"	Order placed within 3 seconds

TC-16	Admin Login	Admin credentials	Admin dashboard displayed	Login completes within 2 seconds
TC-17	Add Product (Admin)	Fill product form	Product added to catalog	Product appears in catalog
TC-18	Update Order Status (Admin)	Change status to "Shipped"	Order status updated, email sent	Status updates within 2 seconds
TC-19	View Order History	Access profile orders	List of user orders displayed	Orders load within 3 seconds
TC-20	Mobile Responsiveness	Mobile viewport	Layout adjusts, touch targets usable	WCAG 2.1 compliance

4.6 Deployment Configuration

4.6.1 Environment Variables

```
# Backend .env
PORT=5000
MONGODB_URI=mongodb://localhost:27017/laptop-store
JWT_SECRET=your_jwt_secret_key
PAYPAL_CLIENT_ID=your_paypal_client_id
PAYPAL_CLIENT_SECRET=your_paypal_client_secret
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
EMAIL_HOST=smtp.gmail.com
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_app_password

# Frontend .env
VITE_API_URL=http://localhost:5000/api
VITE_PAYPAL_CLIENT_ID=your_paypal_client_id
```

4.7 Security Testing Results

Test Area	Result
JWT Authentication	Tokens properly validated on protected routes, 7-day expiration

Password Hashing	bcrypt with 10 salt rounds, no plaintext storage
HTTPS Encryption	All communications encrypted in production
XSS Prevention	Input sanitization on all user inputs
SQL/NoSQL Injection	Mongoose schema validation prevents injection
Role-Based Access Control	Admin routes protected with middleware

4.8 Disaster Recovery Procedures

1. **Daily Database Backup:** Automated MongoDB backup to cloud storage
2. **Log File Rotation:** Weekly compression, deletion of logs older than 30 days
3. **Inventory Sync:** Daily product stock level verification
4. **Security Updates:** Monthly dependency updates and security patches
5. **Performance Monitoring:** Continuous API response time and error rate monitoring
6. **Payment Reconciliation:** Daily PayPal transaction verification

4.9 Conclusion

The MERN Laptop E-commerce Platform successfully implements a comprehensive online store with specialized laptop features. The system leverages the full MERN stack to deliver a scalable, maintainable, and performant e-commerce solution. Key features include JWT-based authentication, advanced product filtering by technical specifications, real-time shopping cart management, PayPal and COD payment integration, and a complete admin dashboard for business management. The platform meets all specified functional and non-functional requirements and is ready for deployment.

End of Document